V Vimalnath Sales Corporation app

Backend plan: modules and submodules

What the server behind the Vimalnath mobile app and admin panel needs to do, split into 12 modules and 76 submodules, with the data each one stores, its proposed API, and the order to build them in.

Prepared for	Vimalnath Sales Corporation
Prepared by	4AM Global Media
Date	7 November 2026
Status	Draft 1.1 for review. Items marked “proposed” need your decision (section 17).

CONTENTS

What's in this document

Overview

1. Why the app needs a backend
2. How the pieces fit together
3. The 12 modules at a glance

Modules

4. Module 1 · Core platform
5. Module 2 · Staff accounts and permissions
6. Module 3 · App users
7. Module 4 · Catalogue
8. Module 5 · Media and files
9. Module 6 · Orders
10. Module 7 · Customers
11. Module 8 · Calculator settings
12. Module 9 · App content and settings
13. Module 10 · Notifications
14. Module 11 · App sync (offline support)
15. Module 12 · Activity log and reports

Plan

16. Build phases and what "done" means
17. Decisions needed before building
18. Proposed technology
19. Possible later modules

How to read the module pages

Each module page has the same parts:

- **Today**: how the app or admin handles this now, which is why the module is needed.
- **Submodules**: numbered (for example 6.2) so they can be referred to in planning and tasks. **Used by** says whether the mobile **App**, the **Admin** panel, or only the **Server** uses it.
- **Phase**: when it gets built. P1 admin edits reach the app, P2 orders reach the shop, P3 notifications, P4 the rest.
- **Data**: the main tables and their key fields.
- **API**: the proposed endpoints. App endpoints start with `/v1/app`, admin endpoints with `/v1/admin`.
- **Rules**: behaviour that must hold, such as how order numbers are given out.

SECTION 1

Why the app needs a backend

Today the mobile app is self-contained: all of its data ships inside the app, and everything people do in it stays on their phone. The admin panel exists as a working preview, but it isn't connected to anything yet.

AREA	TODAY	WITH THE BACKEND
Products and prices	All 650 products and their MRPs are built into the app. Changing one price means releasing a new app version.	Staff change prices in the admin; phones pick up the change the next time they open the app.
Orders	Saved only on the phone that placed them. The shop never receives them. Each phone numbers its own orders from VSC-0001, so numbers clash.	Orders reach the shop instantly with a unique number, and the customer sees the real status.
Customer details	The User screen (name, business, phone, email, city) is saved only on the phone.	Profiles are stored centrally and linked to orders.
Order status	Every order shows "Placed" forever.	New → Confirmed → Dispatched → Delivered, with notifications.
Price lists	Meant to be Google Drive links; the 3 links are still empty, so the app shows "coming soon".	PDFs uploaded in the admin and served from the backend.
Installation videos	YouTube search links, not specific videos.	Real video links, managed in the admin.
Bar sections	10 sample records carried over from the web prototype.	Vimalnath's real profiles, managed in the admin.
Admin panel	Preview only: saves to the browser, accepts any password, orders are sample data.	Real staff accounts and real data, shared by everyone.

What stays on the phone

These work today without internet, and should keep doing so. The backend supplies their data but doesn't take them over:

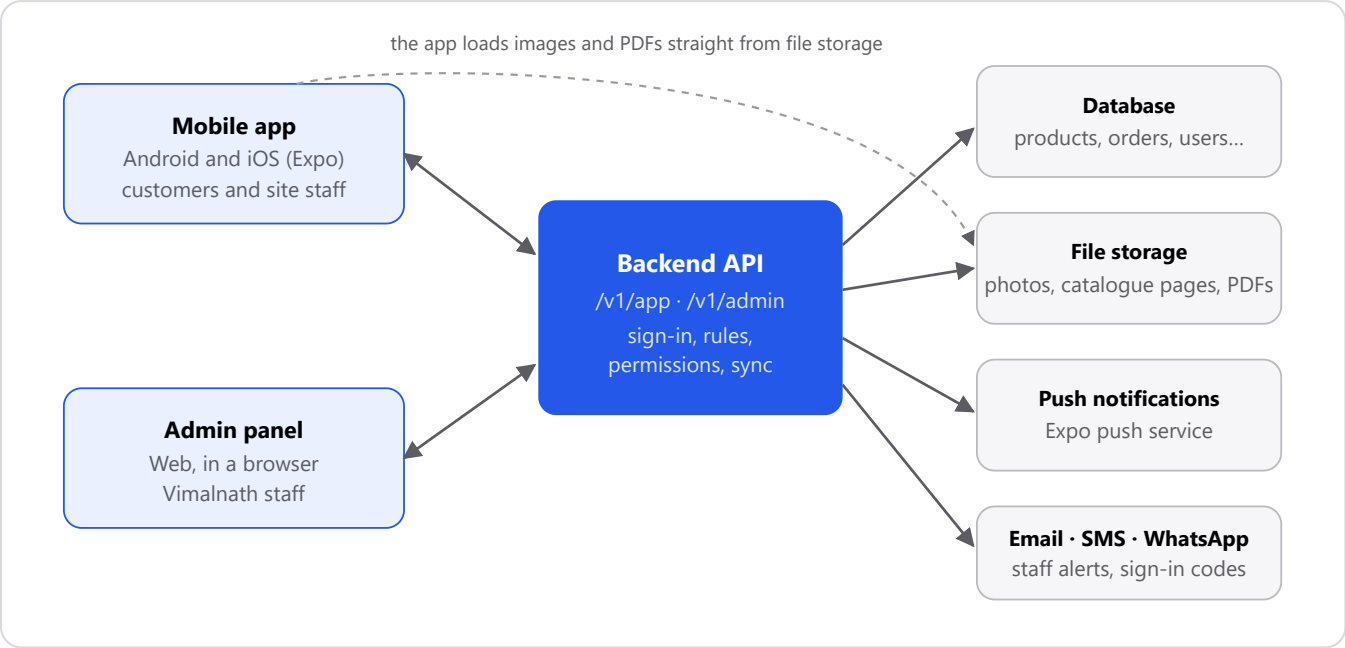
- **Glass calculator** (Cutlist tab): cutting size, glass size and material list.
- **Bar optimizer**: the cut plan and waste figures.
- **Cart**: items waiting to be ordered.
- **PDF export**: order, glass plan and bar plan PDFs shared from the phone.
- **Size unit and last calculator inputs**: small per-phone conveniences.

> The app must keep working with no signal, for example on a building site. Module 11 (App sync) is what makes that possible once the data comes from the server.

SECTION 2

How the pieces fit together

The mobile app and the admin panel both talk to one API. Only the API can change the database and file storage, and it sends notifications through outside services. The app loads images and PDFs straight from file storage, which is faster.



Two kinds of API

App API (`/v1/app/…`)

- Used by the mobile app.
- Mostly reading: catalogue, content and calculator data through sync.
- Writing only for the profile, orders and push tokens.
- Identified by device, and by phone number if OTP sign-in is chosen.

Admin API (`/v1/admin/…`)

- Used by the admin panel.
- Reads and writes everything the staff manage.
- Every request needs a signed-in staff member, and their role must allow the action.
- Every change goes into the activity log.

> The admin panel was built so that its screens only talk to one data layer (`admin/src/data/store.tsx`). Connecting it to this API means replacing that one file, not rewriting the screens.

SECTION 3

The 12 modules at a glance

#	MODULE	PURPOSE	USED BY	PHASES	SUBS
1	**Core platform**	API server, database, security, backups and deployment	Server	P1	7
2	**Staff accounts and permissions**	Real admin sign-in and roles	Admin	P1 P2 P4	6
3	**App users**	Who uses the app: device, profile, optional phone sign-in	App, Admin	P2 P3	6
4	**Catalogue**	Products, options and prices, specs, categories, catalogue pages, price lists	App, Admin	P1 P2	9
5	**Media and files**	Image and PDF upload, storage, thumbnails	Admin, Server	P1 P2 P4	5
6	**Orders**	Orders from the app, numbering, status, notes, documents	App, Admin	P2	10
7	**Customers**	Customer records built from users and orders	Admin	P2	6
8	**Calculator settings**	Cutlist systems, configurations, formula numbers, bar sections	App, Admin	P1 P4	6
9	**App content and settings**	Banners, videos, company details, More Apps, update control	App, Admin	P1 P4	7
10	**Notifications**	Push to customers, alerts to staff	App, Admin	P3 P4	5
11	**App sync**	Delivers data to the app and keeps it working offline	App, Server	P1	5
12	**Activity log and reports**	Who changed what, dashboard figures, exports	Admin	P2 P4	4

76 submodules in total.

How the modules depend on each other

- **Core platform (1)** comes first; everything else runs on it.
- **Staff accounts (2)** protect every admin endpoint, so they come before any admin editing.
- **Media (5)** is needed by Catalogue (4), Calculator settings (8) and App content (9), because they all have pictures or PDFs.
- **App sync (11)** delivers Catalogue (4), Calculator settings (8) and App content (9) to phones.
- **Orders (6)** need Catalogue (4) for product details and App users (3) to know who ordered.
- **Customers (7)** are built from App users (3) and Orders (6).
- **Notifications (10)** react to Orders (6) and need push tokens from App users (3).
- **Activity log (12)** records changes from every admin module.

1 Core platform

The foundation every other module runs on. It has no screens of its own.

USED BY	PHASE	DEPENDS ON
Server	P1, first	Nothing

Today There is no server. The app reads files bundled inside it; the admin reads a JSON file generated from the app's data and saves edits in the browser.

SUBMODULES

#	SUBMODULE	WHAT IT DOES	USED BY	PHASE
1.1	**API server**	One web API with versioned addresses (`/v1/…`) so the app can keep working while newer versions are added later.	Server	P1
1.2	**Database and migrations**	The tables, controlled schema changes, and a one-time import of the app's current data: 650 products, 11 categories, 3 price lists, 3 cutlist systems with 23 configurations, 10 bar sections, 4 home banners and the installation videos.	Server	P1
1.3	**Validation and errors**	Every input is checked on the server, whatever the app or admin already checked. Errors come back in one format with a clear message and the field it's about.	Server	P1
1.4	**Configuration and secrets**	Separate test and live settings. Database passwords and service keys live in the hosting provider's secret store, never in the code.	Server	P1
1.5	**Security**	HTTPS only; limits on repeated requests (stops password guessing and spam orders); only the admin's own web address may call the admin API; request logs.	Server	P1
1.6	**Backups and monitoring**	Automatic daily database backups with restore tested; alerts when the server is down or errors jump.	Server	P1
1.7	**Deployment**	Automatic build and release; a staging copy for testing changes before they reach the live app.	Server	P1

DATA

schema_migrations	Which database changes have been applied, so every environment matches.

API

GET	`/v1/health`	Is the server up and can it reach the database. Used by monitoring.

RULES

- All amounts are stored as whole rupees, as the app uses them today.
- All times are stored in UTC and shown in Indian time (IST) in the app and admin.
- Deleting anything asks for confirmation first and is recorded in the activity log (Module 12). Hiding is offered as the safer choice where it exists.

2 Staff accounts and permissions

Real sign-in for the admin panel, and control over who can change what.

USED BY	PHASE	DEPENDS ON
Admin	P1 (2.5 in P2, 2.6 in P4)	Core platform

Today The admin preview accepts any email and password, and everyone can change everything.

SUBMODULES

#	SUBMODULE	WHAT IT DOES	USED BY	PHASE
2.1	**Sign in and sign out**	Email and password; a secure session that ends after a period of no use. Works with password managers.	Admin	P1
2.2	**Password reset and change**	"Forgot password" sends a one-time link by email; staff can change their password in Settings.	Admin	P1
2.3	**Staff management**	The Owner invites staff by email, edits their name and role, and deactivates people who leave (their history stays).	Admin	P1
2.4	**Roles and permissions**	Each staff member has one role. The server checks the role on every action, so hiding a button is never the only protection.	Admin	P1
2.5	**Login protection**	Temporary lock after repeated wrong passwords; optional two-step sign-in for the Owner.	Admin	P2
2.6	**Active sessions**	See which devices you're signed in on and sign the others out.	Admin	P4

PROPOSED ROLES (TO CONFIRM, QUESTION 3)

AREA	OWNER	SALES	CATALOGUE EDITOR
Dashboard and reports	**Full**	View	View
Orders	**Full**	**Full**	View
Customers	**Full**	**Full**	None
Products, prices, categories, price lists	**Full**	View	**Full**
Calculator settings	**Full**	View	**Full**
App content (banners, videos)	**Full**	None	**Full**
Settings, update control, staff accounts	**Full**	None	None
Activity log	**Full**	Own changes	Own changes

DATA

staff_users	id, name, email, password hash, role, active, last sign-in
staff_sessions	id, staff member, device, created, last used, expires
password_resets	staff member, one-time token (stored hashed), expires, used

API

POST	`/v1/admin/auth/sign-in · sign-out`	Start or end a session.
POST	`/v1/admin/auth/forgot-password · reset-password`	Email a reset link (always says "sent", so it doesn't reveal who has an account); set the new password.
GET	`/v1/admin/me`	The signed-in staff member and their role.
GET · POST · PATCH	`/v1/admin/staff, /v1/admin/staff/:id`	List, invite, change role, deactivate.

3 App users

Who is using the app, so orders and notifications reach the right person.

USED BY	PHASE	DEPENDS ON
App, Admin	P2 (3.5 in P3)	Core platform

Today There's no sign-in. The User screen asks for name, business, phone, email and city and keeps them only on the phone. The size unit (mm or inches) is also saved per phone.

SUBMODULES

#	SUBMODULE	WHAT IT DOES	USED BY	PHASE
3.1	**Device registration**	The first time the app opens online, it gets an ID for that installation and reports its app version and platform (Android or iOS). No personal details needed.	App	P2
3.2	**Profile**	The User screen's fields saved on the server too, so they're prefilled on orders and survive reinstalling the app (with phone sign-in).	App	P2
3.3	**Phone sign-in (optional)**	A one-time code by SMS proves the phone number. Lets a customer see their orders on a new phone. See question 1.	App	P2
3.4	**Preferences**	Size unit and any later per-user settings.	App	P2
3.5	**Push token**	The address used to send this phone notifications; refreshed whenever it changes.	App	P3
3.6	**App users list**	Staff see users, when they were last active and which app version they run; they can block misuse (for example fake orders).	Admin	P2

DATA

app_devices	id, platform, app version, push token, user (if any), first seen, last seen, blocked
app_users	id, phone, verified at, name, business, email, city, size unit, created
otp_codes	phone, code (stored hashed), expires, attempts (only with phone sign-in)

API

POST	`/v1/app/devices`	Register this installation; returns a device token the app sends with later requests.
PUT	`/v1/app/profile`	Save the profile and preferences.
POST	`/v1/app/auth/request-code`	Send an SMS code to a phone number (phone sign-in only).
POST	`/v1/app/auth/verify-code`	Check the code and link the device to that user.
PUT	`/v1/app/push-token`	Save or refresh the push token.
GET	`/v1/admin/app-users`	List app users with search and filters.
PATCH	`/v1/admin/app-users/:id`	Block or unblock.

RULES

- Browsing the catalogue and using the calculators never require sign-in.
- SMS codes expire after a few minutes and allow only a few attempts.
- Only what's needed is collected; the app's Privacy information must list it.

4 Catalogue

Products and prices, edited in the admin and shown in the app's Shop without an app update.

USED BY	PHASE	DEPENDS ON
App, Admin	P1 (4.8 and 4.9 in P2)	Core, Staff accounts, Media

Today 650 products are built into the app from the Master, Office and Tavic catalogues. 632 have an exact MRP; the rest send people to their catalogue page. Every price change needs a new app release.

SUBMODULES

#	SUBMODULE	WHAT IT DOES	USED BY	PHASE
4.1	**Products**	Code, name, section, category, photo, shown or hidden, featured, order within its category.	Admin, App	P1
4.2	**Options and prices**	Each orderable choice: label (for example "Heavy Duty · Black Matte (BM)"), option code, MRP.	Admin, App	P1
4.3	**Specifications**	Label and value pairs such as max panel size, weight capacity, glass thickness, profile.	Admin, App	P1
4.4	**Categories**	The 11 Shop categories (Glass Hardware, Door Control, Telescopic Sliding…): name, image, order, shown or hidden.	Admin, App	P1
4.5	**Catalogue sources and pages**	The Master, Office and Tavic catalogues, their page images, and which page each product is on (the app opens that page for sizes and prices).	Admin, App	P1
4.6	**Price lists**	The 3 price-list PDFs: title, edition (for example Taiton · Feb 2026), file, published or "coming soon".	Admin, App	P1
4.7	**Search and filters**	Search by code or name, filter by category, catalogue, price status, photo status, shown or hidden.	Admin, App	P1
4.8	**Bulk price update**	Upload a CSV or Excel file when a new price-list edition arrives; see every change (old MRP → new MRP, new and missing codes) before applying it in one step.	Admin	P2
4.9	**Price history**	Every MRP change is kept with who made it and when.	Admin	P2

DATA

products	id, code, name, section, category, catalogue source, catalogue page, photo, featured, visible, sort order, updated
product_options	id, product, label, option code, MRP, sort order
product_specs	product, label, value, sort order
categories	id, label, image, image fit (cover or contain), visible, sort order
catalogue_pages	catalogue source, page number, page image
price_lists	id, number, title, kicker, meta line, colour tone, edition, PDF file, published
price_history	option, old MRP, new MRP, changed by, changed at, source (manual edit or bulk import)

API

`GET · POST`	`/v1/admin/products`	List with search and filters; create a product.
`GET · PUT · DELETE`	`/v1/admin/products/:id`	Read, save (with options and specs), delete.

PATCH	/v1/admin/products/bulk	Show or hide, or feature, many products at once.
POST	/v1/admin/products/import	Upload a price file; returns a preview of changes.
POST	/v1/admin/products/import/:id/apply	Apply a previewed import.
GET · PUT	/v1/admin/categories	List; save names, images, order and visibility.
GET · PUT	/v1/admin/price-lists/:id	Read; save details, upload the PDF, publish.

The app doesn't call these. It receives the catalogue through App sync (Module 11).

RULES

- The product id is unique, not the code: 55 codes appear on more than one product, because the catalogues repeat items. A product with no options shows "price in catalogue" and opens its catalogue page, as now.
- Hidden products disappear from the app but stay in old orders exactly as ordered.
- Changing a price never changes an order that was already placed (see 6.3).

5 Media and files

Storing and serving every image and PDF the app and admin use.

USED BY	PHASE	DEPENDS ON
Admin, Server	P1 (5.4 in P2, 5.5 in P4)	Core, Staff accounts

Today Images ship inside the app. Product thumbnails load the full-size photo, about 950 KB each, for a 44-pixel thumbnail.

SUBMODULES

#	SUBMODULE	WHAT IT DOES	USED BY	PHASE
5.1	**Upload**	Images (JPG, PNG, WebP) and PDFs, with type and size checks and a progress bar.	Admin	P1
5.2	**Storage and delivery**	Cloud file storage with a content delivery network, so phones across India download quickly. The app caches what it has downloaded.	Server	P1
5.3	**Thumbnails**	Small, medium and large WebP versions made automatically on upload, so lists load small images and detail screens load sharp ones.	Server	P1
5.4	**Media library**	Browse, search, reuse and replace uploaded files; see where each one is used.	Admin	P2
5.5	**Cleanup**	Find files nothing uses any more and remove them after confirmation.	Server	P4

WHAT IT STORES

Product photos; category images; catalogue page images (102 pages today); price-list PDFs; home banner images; cutlist system photos and configuration pictures; the company logo.

DATA

media_files	id, kind (image or PDF), original name, file type, size, width, height, storage key, thumbnail keys, uploaded by, uploaded at

API

POST	`/v1/admin/media`	Upload a file; returns its id and addresses.
GET	`/v1/admin/media`	Browse and search the library.
DELETE	`/v1/admin/media/:id`	Remove a file (refused while something still uses it).

RULES

- A replaced file gets a new address, so phones never show an outdated cached copy.
- Proposed limits: images up to 5 MB, PDFs up to 25 MB (to confirm against the real price-list sizes).

6 Orders

Orders placed in the app reach the shop, get a unique number, and move through to delivery.

USED BY	PHASE	DEPENDS ON
App, Admin	P2	Catalogue, App users

Today The Place Order sheet asks for a customer or site name, an optional phone and optional notes (delivery address, timing). Each cart line can carry a note. The order is saved only on that phone, numbered from VSC-0001 per phone, and always shows “Placed”.

SUBMODULES

#	SUBMODULE	WHAT IT DOES	USED BY	PHASE
6.1	**Place order**	Sends the cart to the server: for each line the product, option and option code, quantity, MRP and note; plus customer or site name, phone and delivery notes.	App	P2
6.2	**Order numbering**	The server gives out VSC-0001, VSC-0002… in one shared sequence, so no two orders ever share a number.	Server	P2
6.3	**Price snapshot and totals**	Saves each product’s name, code and MRP as they were when ordered. The total adds only lines with an MRP; catalogue-priced lines are listed but left out, as the app does today.	Server	P2
6.4	**Offline orders**	An order placed without signal waits on the phone and is sent when it reconnects. It can never arrive twice.	App	P2
6.5	**Status workflow**	New → Confirmed → Dispatched → Delivered, or Cancelled. Each change records who made it and when.	Admin	P2
6.6	**Order list and search**	Filter by status, product, customer and date; search by order number, name, phone or product code; confirm, dispatch or deliver several at once.	Admin	P2
6.7	**Internal notes**	Staff-only notes on an order, for example quotes for catalogue-priced items. Never shown to the customer.	Admin	P2
6.8	**My Orders**	The app’s Orders tab loads the user’s orders from the server and shows the real status and its history.	App	P2
6.9	**Order sheet**	A printable order and PDF with company details, matching the PDF the app already shares.	Admin, App	P2
6.10	**Export**	CSV of the orders currently shown, for Excel.	Admin	P2

DATA

orders	id, number, app user, device, customer or site name, phone, delivery notes, status, total, client order id (prevents duplicates), placed at
order_lines	order, product, code, name, option label, option code, note, quantity, MRP (empty when catalogue-priced)
order_events	order, status, at, by (the app or a staff member)
order_internal_notes	order, text, edited by, edited at

API

POST	`/v1/app/orders`	Place an order. Carries a client order id so a resend after a dropped connection is recognised.
GET	`/v1/app/orders`	The user’s orders, newest first.
GET	`/v1/app/orders/:id`	One order with its status history.

GET	/v1/admin/orders	List with filters (status, product, customer, dates) and search.
GET	/v1/admin/orders/:id	Full order with lines, history and internal note.
POST	/v1/admin/orders/:id/status	Change status.
POST	/v1/admin/orders/bulk-status	Change the status of several orders.
PUT	/v1/admin/orders/:id/internal-note	Save or remove the internal note.
GET	/v1/admin/orders/export.csv	CSV of the filtered list.
GET	/v1/admin/orders/:id/sheet.pdf	Order sheet PDF.

RULES

- Order numbers come from one database sequence inside the same transaction that saves the order, so two orders can't get the same number even at the same moment.
- The server re-checks every line against the catalogue: unknown or hidden products are refused with a clear message. The MRP saved is the server's current MRP, not whatever the phone sent.
- Orders are never deleted. Mistakes are cancelled, and a cancellation can be reversed by changing the status back (recorded in the history).
- Orders already on phones from before the update stay on those phones; they're not uploaded (to confirm, question 5).

7 Customers

One record per customer, so staff see everything about who they're dealing with.

USED BY	PHASE	DEPENDS ON
Admin	P2	App users, Orders

Today The admin preview groups sample orders by the customer or site name typed on the order. The app doesn't collect anything else about customers.

SUBMODULES

#	SUBMODULE	WHAT IT DOES	USED BY	PHASE
7.1	**Customer records**	Created automatically from app users and the names and phones on their orders.	Server	P2
7.2	**Duplicate merge**	Suggests records that look like the same customer (same phone number, similar name) and lets staff merge them.	Admin	P2
7.3	**Customer profile**	Contact details, delivery addresses used, customer since, lifetime value, average order, open orders.	Admin	P2
7.4	**History and buying pattern**	All their orders and the products they order most.	Admin	P2
7.5	**Staff notes and tags**	Private notes and tags such as "dealer", "contractor" or "pays late".	Admin	P2
7.6	**Export**	CSV of customers.	Admin	P2

DATA

customers	id, name, business, phone, email, city, tags, staff note, merged into (after a merge), created
customer_links	customer, app user or order (how each order is tied to its customer)

API

`GET`	`/v1/admin/customers`	List with search (name or phone), sorting and totals.
`GET · PATCH`	`/v1/admin/customers/:id`	Profile with history and buying pattern; save notes and tags.
`POST`	`/v1/admin/customers/:id/merge`	Merge another record into this one.
`GET`	`/v1/admin/customers/export.csv`	CSV export.

RULES

- All money figures leave out cancelled orders, matching the admin dashboard.
- A merge moves every order to the surviving record and can be traced in the activity log.

8 Calculator settings

The data behind the Cutlist and Optimizer tabs. The calculations themselves stay on the phone.

USED BY	PHASE	DEPENDS ON
App, Admin	P1 (8.3, 8.4, 8.6 in P4)	Core, Media, App sync

Today 3 cutlist systems with 23 configurations. The glass formulas are written in the app's code: 4 are confirmed by Vimalnath (1+0, 1+1, 2+0, 2+1), 4 reuse the same rule (3+0, 3+1, 4+0, 4+1), and all 15 synchro and folding configurations use a placeholder (opening − 20 mm, split evenly). The 10 bar sections are samples from the old prototype.

SUBMODULES

#	SUBMODULE	WHAT IT DOES	USED BY	PHASE
8.1	**Cutlist systems**	Telescopic Sliding, Synchronized System, Sliding Folding System: name, subtitle, photo.	Admin, App	P1
8.2	**Configurations**	The 23 configurations (1+0 to 4+1 telescopic, 2+0 to 6+2 synchro, 2 to 10 door folding): name, picture, order, formula status.	Admin, App	P1
8.3	**Glass formula numbers**	Overlap per sliding panel (16 mm), height deduction (92 mm), glass deduction (21 mm), height range (1,800–2,998 mm), minimum width per panel (600 mm), maximum width per system. Editable without an app update.	Admin, App	P4
8.4	**Material list rules**	Vertical length switch (2.5 m up to 2,489 mm, then 3 m), horizontal and top-track rules per system, kit names, gaskets (3 per panel) and connectors (4 per panel).	Admin, App	P4
8.5	**Bar sections**	Code, name, system, standard bar length, profile size, description. Replaces the 10 prototype samples with Vimalnath's real profiles.	Admin, App	P1
8.6	**Optimizer defaults**	Standard bar lengths and cutting loss the Bar Optimizer starts with (cutting loss is 0 mm today).	Admin, App	P4

DATA

cutlist_systems	id (telescopic, synchronized, folding), name, subtitle, photo, sort order
cutlist_configurations	id, system, name, picture, sort order, formula status (confirmed, same rule, placeholder)
formula_parameters	key, value, unit, applies to (all systems or one configuration), changed by, changed at
bar_sections	code, name, system, bar length (mm), profile size, description, sort order
optimizer_defaults	standard bar lengths, cutting loss

API

`GET` · `PUT`	`/v1/admin/cutlist/systems/:id, …/configurations/:id`	Read and save system cards and configurations.
`GET` · `PUT`	`/v1/admin/formula-parameters`	Read and save formula numbers (Phase 4).
`GET` · `POST` · `PUT` · `DELETE`	`/v1/admin/bar-sections, …/:code`	List, add, save and delete sections.

RULES

- Formulas stay in the app's code; in Phase 4 only their numbers become editable.
- Changing a number shows a before-and-after preview first, because a wrong number means glass cut to the wrong size.
- The real formulas for the 15 placeholders and the real bar sections must come from Vimalnath (question 6).

9 App content and settings

Everything else the app shows, and app-wide switches.

USED BY	PHASE	DEPENDS ON
App, Admin	P1 (9.7 in P4)	Core, Media, App sync

Today 4 home banners share one headline. Installation videos are YouTube search links, not specific videos. About shows the app version and developer. More Apps says “coming soon”. The app has no company contact details (phone, email, website, address) yet.

SUBMODULES

#	SUBMODULE	WHAT IT DOES	USED BY	PHASE
9.1	**Home banners**	Image, small heading, headline, text, order, on or off, for the home slideshow.	Admin, App	P1
9.2	**Installation videos**	Sections and videos: title, detail, YouTube link, order.	Admin, App	P1
9.3	**Company details**	Company name, tagline, phone, email, website, address, logo; shown on About and on order sheets.	Admin, App	P1
9.4	**More Apps**	Other 4AM Global Media apps: name, description, icon, store links.	Admin, App	P1
9.5	**App defaults**	Default size unit for new users, and the price-list edition shown in the app.	Admin, App	P1
9.6	**Update control**	Latest and minimum app versions. Below the minimum, the app asks people to update before ordering (needed if an old app version could send bad orders).	Admin, App	P1
9.7	**Announcement**	An optional notice at the top of the app, with start and end dates (holiday closure, new price list).	Admin, App	P4

DATA

banners	id, image, kicker, headline, body, active, sort order
video_sections · videos	section title and order; video title, detail, link, order
more_apps	name, description, icon, Play Store link, App Store link, order
settings	company details, default unit, price-list edition, latest and minimum app version, announcement

API

`GET` · `PUT`	`/v1/admin/banners`	List; save all banners and their order.
`GET` · `PUT`	`/v1/admin/videos`	List; save sections and videos.
`GET` · `PUT`	`/v1/admin/more-apps`	List; save.
`GET` · `PUT`	`/v1/admin/settings`	Read; save company details and app settings.

10 Notifications

Tell customers when their order moves on, and tell staff when a new order arrives.

USED BY	PHASE	DEPENDS ON
App, Admin	P3 (10.5 in P4)	App users (push token), Orders

Today No notifications. The app doesn't include push notifications yet (no `expo-notifications`), so an app update is part of this phase.

SUBMODULES

#	SUBMODULE	WHAT IT DOES	USED BY	PHASE
10.1	**Customer push**	For example "Your order VSC-0042 has been dispatched". Tapping it opens that order in the app.	App	P3
10.2	**Staff alerts**	Email to staff on each new order. WhatsApp too if a WhatsApp Business account is set up (question 4).	Admin	P3
10.3	**Message templates**	The wording for each event, editable by the Owner.	Admin	P3
10.4	**Delivery log**	What was sent, to whom, when, and whether it failed.	Admin	P3
10.5	**Broadcast (optional)**	Announce a new product or price list to every app user.	Admin	P4

EVENTS THAT NOTIFY

EVENT	CUSTOMER	STAFF
Order placed	Shown in the app straight away (no push needed)	Email (and WhatsApp if set up)
Order confirmed	Push	—
Order dispatched	Push	—
Order delivered	Push	—
Order cancelled	Push	—

DATA

notification_templates	event, channel (push, email, WhatsApp), title, body
notification_log	event, channel, recipient, order, status (sent or failed), error, sent at
staff_alert_recipients	email addresses and WhatsApp numbers that get new-order alerts

API

`GET` · `PUT`	`/v1/admin/notifications/templates`	Read and edit message wording.
`GET`	`/v1/admin/notifications/log`	Delivery log with filters.
`POST`	`/v1/admin/notifications/broadcast`	Send an announcement (Phase 4).

RULES

- Notifications are sent in the background. A slow or failed notification never blocks saving an order or changing a status.
- Failed sends are retried a few times, then logged as failed.

11 App sync (offline support)

Delivers the catalogue, content and calculator data to phones, and keeps the app working without signal.

USED BY	PHASE	DEPENDS ON
App, Server	P1	Catalogue, Calculator settings, App content

Today Everything is built into the app, so it works fully offline but can only change with an app release.

SUBMODULES

#	SUBMODULE	WHAT IT DOES	USED BY	PHASE
11.1	**Content version**	One number that goes up every time staff change anything the app shows.	Server	P1
11.2	**Changes since**	The app sends the version it has; the server returns only what changed or was removed since then.	App	P1
11.3	**Full download**	Everything at once, on first launch or when the phone is too far behind.	App	P1
11.4	**Saved copy on the phone**	The app keeps the last download and works from it without signal.	App	P1
11.5	**Built-in fallback**	The app still ships with a copy of the data, so the very first launch works offline too.	App	P1

WHAT GETS SYNCED

Products with options and specs; categories; catalogue pages; price lists; cutlist systems and configurations; bar sections; formula numbers (Phase 4); banners; installation videos; More Apps; company details and app settings.

HOW IT BEHAVES

1. The app opens and shows its saved copy immediately; it never waits for the network.
2. In the background it asks the server for changes since its version.
3. If there are changes, it saves them and updates the screens; images download as they're needed.
4. It checks again whenever the app comes back to the foreground, at most every few minutes.

DATA

content_changes	version, what changed (kind and id), changed or removed, at

API

GET	`/v1/app/sync?since=<version>`	Changes and removals since that version, plus the new version number.
GET	`/v1/app/bootstrap`	The full data set and its version.

RULES

- The Shop never shows a half-updated catalogue: a set of changes is saved on the phone all together or not at all.
- Prices shown offline may be out of date; the order rules in 6.3 make sure the saved MRP is always the server's.

12 Activity log and reports

A record of every change, and the figures behind the admin dashboard.

USED BY	PHASE	DEPENDS ON
Admin	P2 (12.2–12.4 in P4)	Every admin module, Orders

Today The admin preview already has an Activity page and a dashboard, working from data saved in the browser and sample orders.

SUBMODULES

#	SUBMODULE	WHAT IT DOES	USED BY	PHASE
12.1	**Activity log**	Who changed what and when, with the values before and after. Covers orders, catalogue, calculators, content and settings.	Admin	P2
12.2	**Dashboard figures**	Order value, orders and average order with change against the previous period; orders per day; open orders by status; time to confirm, dispatch and deliver; top products and customers; sales by category.	Admin	P4
12.3	**Reports and exports**	CSV for orders, customers, products and prices, and price history.	Admin	P4
12.4	**Catalogue health**	Products without a price or photo, unpublished price lists, placeholder formulas, sections still on sample data.	Admin	P4

DATA

activity_log	id, at, staff member, area, action, item kind, item id, summary, values before, values after

API

GET	`/v1/admin/activity`	The log with filters (area, staff member, dates) and search.
GET	`/v1/admin/reports/dashboard?range=30`	Every dashboard figure for 7, 30 or 90 days.
GET	`/v1/admin/reports/catalogue-health`	Counts and lists for the “Needs attention” panel.

RULES

- The log can’t be edited or deleted from the admin.
- Money figures are at MRP and leave out cancelled orders and catalogue-priced lines, as labelled on the dashboard.

SECTION 16

Build phases and what “done” means

Each phase ends with something Vimalnath can use, and includes the app and admin changes needed alongside the backend.

Phase 1 Admin edits reach the app

Backend: Modules 1, 2, 4 (4.1–4.7), 5 (5.1–5.3), 9 (9.1–9.6), 11, and 8.1, 8.2, 8.5.

Admin panel: real sign-in and staff management; connect Products, Categories, Price Lists, Cutlist Systems, Bar Sections, Banners, Videos and Settings to the API; real uploads.

App: read data through sync with a saved copy; check the update-control version; load images from the server.

Done when: a price changed in the admin shows in the app the next time it’s opened, with no app release; the app still works in flight mode; staff sign in with their own accounts.

Phase 2 Orders reach the shop

Backend: Modules 3 (except 3.5), 6, 7, 12.1, plus 2.5, 4.8, 4.9 and 5.4.

Admin panel: connect Orders, Customers and Activity to the API; add App Users, bulk price update and the media library.

App: register the device; save the profile (and phone sign-in if chosen); send orders, queueing them when offline; Orders tab reads from the server.

Done when: an order placed in the app appears in the admin within seconds with a unique number; a status change in the admin shows in the app; an order placed offline arrives once when the phone reconnects.

Phase 3 Notifications

Backend: Module 10 (10.1–10.4) and 3.5.

App: add push notifications (`expo-notifications`), ask permission at a sensible moment (after the first order), open the order when a notification is tapped.

Done when: a customer’s phone gets a notification within a minute of a status change, and staff get an email for every new order.

Phase 4 Calculators, reports and extras

Backend: 8.3, 8.4, 8.6, 12.2–12.4, 9.7, 10.5, 2.6, 5.5.

App: calculators read formula numbers and optimizer defaults from synced data; show the announcement.

Done when: a formula number changed in the admin changes the app’s cutting sizes after sync, and the dashboard runs on real orders.

SECTION 17

Decisions needed before building

Questions 1 to 3 are needed before Phase 1 starts. The others can be answered during Phase 1.

1. Should customers sign in with their phone number?

With an SMS code (OTP), a customer keeps their orders when they change phones, and fake orders are harder. Without it, the profile is filled in once and orders are tied to the phone. OTP adds a per-message SMS cost and one extra step. Affects 3.3 and 6.8.

2. Hosting and budget

Is there a hosting provider, cloud account or monthly budget to work within? Servers in India (Mumbai) are recommended for speed. Affects Module 1 and section 18.

3. Who will use the admin?

How many staff, and do the three proposed roles (Owner, Sales, Catalogue editor) fit how Vimalnath works? Affects 2.4.

4. Staff alerts by WhatsApp?

Email alerts are simple. WhatsApp alerts need a WhatsApp Business API account and approved message templates. Affects 10.2.

5. Orders already on phones

Should orders placed before the update stay on those phones (simplest), or be uploaded once? Affects 6.1.

6. Real calculator data

Who will supply the confirmed formulas for the 15 synchro and folding configurations, and the real bar sections with their codes and lengths? Affects 8.3–8.5.

7. Price-list PDFs

The three current PDFs (Master, Office Partition, Tavic) are needed to upload in Phase 1. Affects 4.6.

8. Payments and GST invoices

The app takes neither today. Should they stay out of this plan? See section 19.

SECTION 18

Proposed technology

A suggestion to confirm with question 2. It keeps one language across the app, admin and server, so the same people can work on all three and share code such as the product and order types.

PART	PROPOSED	WHY
Server language	TypeScript on Node.js	Same language as the Expo app and the admin panel; types for products and orders can be shared.
API framework	Hono or Fastify	Small, fast and well suited to a JSON API.
Database	PostgreSQL	Reliable for orders and money; a sequence gives safe order numbers; good reporting queries.
Database access	Drizzle ORM with migrations	Typed queries and controlled schema changes.
File storage	S3-compatible storage with a CDN (for example Cloudflare R2 or AWS S3)	Cheap storage for images and PDFs with fast delivery across India.
Push notifications	Expo push service	Works directly with the Expo app on Android and iOS.
Email	A transactional email service	Password resets and new-order alerts.
SMS (if OTP chosen)	An Indian SMS provider with DLT registration	Indian SMS rules require registered sender IDs and templates.
Hosting	A managed host with a Mumbai region	Low delay for users in India; managed backups.

SECTION 19

Possible later modules

Not in the app today and not part of this plan. Each would be its own module if Vimalnath wants it.

MODULE	WHAT IT WOULD ADD
Payments	Pay online (UPI, cards) when ordering, or record advance payments against an order.
GST invoicing	Tax invoices with GSTIN, HSN codes and tax breakdown, generated from orders.
Stock and inventory	Stock levels per product, low-stock alerts, “out of stock” in the app.
Dealer pricing	Special prices or discounts for dealers and contractors instead of MRP only.
Quotations	Turn an order into a priced quote, including catalogue-priced items, and send it to the customer.
Dispatch tracking	Vehicle, driver and delivery proof for each dispatch.
Branches	More than one shop or warehouse, each with its own orders and staff.
Saved calculations	Glass and bar plans saved to the account, so they can be reopened on another phone or attached to an order.

Figures in this document come from the current app and admin code as of 7 November 2026: 650 products (632 with an exact MRP), 11 categories, 3 price lists, 102 catalogue page images, 3 cutlist systems with 23 configurations, 10 bar sections and 4 home banners.